Exploring RNNs Through Stock Price Prediction

Raul Sebastien B. Baldemoro, Jan Vincent G. Elleazar

Department of Computer Science, College of Information and Computing Sciences, University of Santo Tomas España Blvd, Sampaloc, Manila, 1008 Metro Manila, Philippines

raulsebastian.baldemoro.cics@ust.edu.ph
janvincent.elleazar.cics@ust.edu.ph

I. Introduction

Financial market time-series prediction is still a difficult research problem because asset prices are non-stationary, noisy, and vulnerable to both exogenous shocks and hidden temporal structures. Past linear models (e.g., ARIMA) are not capable to model non-linear relationships and long-range dependencies that often occur in equity prices. Deep learning, in the form of Recurrent Neural Networks (RNNs) and their gated descendants, has proved to be a promising candidate, offering a way to handle variable-length sequences while keeping track of long-term context through recurrent connections [1].

Of RNN architectures, LSTM and GRU cells have shown robust empirical performance in financial forecasting tasks by avoiding vanishing-gradient problems and selectively retaining meaningful information over time steps [2]. Recent research using LSTM-based models on stocks like Amazon (AMZN) provides accuracy improvements over shallow networks and traditional statistical baselines, illustrating the applied value of gated recurrence for stock dynamics modeling [3]. Supporting research still refines such architectures—e.g., hybrid LSTM-CNN pipelines, symbolic feature augmentation, and adaptive optimisers—aiming to leverage domain-specific structure in price series [4].

Even with these improvements, prior work mostly optimizes models for headline accuracy and has a tendency to solve network design decisions as constants. This lab exercise takes an experimental stance: students implement an RNN from the ground up and experimentally manipulate hyper-parameters, depth in architecture, input representation, and training protocols to see how each of these affects forecasting quality. The idea is that exhaustive exploration—rather than aiming for margin accuracy—provides more insight into why specific configurations work or do not work on sequential financial data.

A. *Dataset*

The experiments utilize a single provided historical price file, "*AMZN_2006-01-01_to_2018-01-01.csv*", with 3001 daily Amazon.com, Inc. observations. The file has Open, High, Low, Close, Adjusted Close, and Volume columns, and a Date index. Targeting this 12-year window sidesteps look-ahead bias and is adequate temporal depth to test short- and long-sequence models without being computationally intensive for classroom hardware.

II. Methodology

This section explains the end-to-end workflow followed in the laboratory, from raw data ingestion to model evaluation.

A. Data Preprocessing

The raw dataset is first read into a pandas DataFrame with the Date column parsed as a datetime index and sorted in ascending order to preserve temporal integrity. Pre-processing then proceeds through the following pipeline:

1. Target selection – We forecast the daily High price. All other columns are discarded unless an experiment explicitly engineers additional features (e.g., lagged prices or moving averages).

2. Chronological split – The series is partitioned without shuffling into 70 % training, 15 % validation, and 15 % hold-out test sets, ensuring that the model never peeks into the future.

3. Normalisation – A *MinMaxScaler* is fitted on the training set alone and applied uniformly to validation and test data. Scaling to the [0,1] range accelerates convergence and keeps gradients numerically stable.

4. Sliding-window construction – For a chosen look-back length L (baseline L=60 days), we slide a window across the scaled series to generate supervised samples:

$$Xi = [pi - L, \ldots, pi - 1], yi = pi.$$

The procedure is repeated for each split, yielding tensors of shape (samples, L, features) ready for an RNN.

5. Feature engineering (experiment-specific) – Some experiments append extra columns—such as p_{i-1}(lag-1), a 5-day moving average, or daily price deltas—*before* scaling so that every feature shares the same transformation parameters.

B. *Baseline Architecture*

Layer	Configuration
Input	Tensor of shape *(60, 1)* – 60 timesteps, 1 feature ('High')
LSTM-1	5 units, *return_sequences=True*
LSTM-2	5 units
Dense-out	1 unit, linear activation

The baseline has a purposefully limited network size to highlight the incremental effects of the following changes.

C. *Experimental Factors*

Ten experiment families modify one dimension at a time:

1. Pre-processing variants – z-score normalisation, log scaling.

2. Depth – add 1–2 extra LSTM layers or a Dense bottleneck.

3. Cell type – swap LSTM with SimpleRNN or GRU.

4. Hidden units – ±100 % relative to baseline.

5. Learning-rate strategy – LR $\in \{0.01, 0.001, 0.0001\}$; cosine decay scheduler.

6. Sequence length – $L \in \{20, 60, 120\}$.

7. Activation functions – ReLU vs. sigmoid inside recurrent layers and output.

8. Regularisation – dropout 0.2 placed after individual or all LSTM layers, plus input-dropout.

9. Optimiser – Adam, SGD (momentum = 0.9), Adagrad.

10. Lagged-feature engineering – Experiments 10.1–10.4 introduce lag-1, 5-day MA, daily change, and a combined 3-feature tensor.

All changes are orthogonal; each experiment alters only one axis so causal impact is traceable.

D. *Training Protocol*

All experiments have the same training recipe with the exception of a particular experiment overwriting a specific item:

- Loss function – Models are optimized to minimize Mean-Squared Error (MSE), the standard default for real-valued regression targets such as stock prices.

- Optimiser – RMSProp with learning-rate base of 0.001 achieves convergent

performance on noisy financial data.

- Batch size – Mini-batches of 32 sequences offer a compromise between gradient stability and GPU/CPU memory constraints common in classroom hardware.

- Epoch schedule – 10 training epochs per experiment is sufficient for its horizon. It is long enough to reveal learning trends but short enough that a small network does not become strongly over-fit; longer, more high-capacity experiments may safely go longer if early-stopping is not activated.

- Early stopping – Val_loss is monitored during training; optimisation stops and best weights are saved if validation error is not increasing for 3 successive epochs.

- Weight initialization & reproducibility – Keras defaults are kept, and *tf.random.set_seed(42)* is called prior to rlunning so that results can be reproduced on other machines and re-rlun.

These baseline settings ensure that differences observed in later sections derive from the experimental factor under test rather than from inconsistent training hyper-parameters.

E. Evaluation Metrics

- Root Mean Squared Error (RMSE) – primary metric reported on validation and test splits.
- Mean Absolute Percentage Error (MAPE) – complementary scale-free indicator.
- Training curves – *loss* and *val_loss* vs. epoch, used qualitatively to judge stability and over/under-fitting.

F. Implementation Environment

Model performance is evaluated mainly in terms of Root Mean Squared Error (RMSE). RMSE estimates the square root of the mean squared differences between actual and predicted prices, maintaining the original scale of the target variable and penalizing bigger errors more severely than smaller ones. During the course of the research, RMSE is calculated for both the tuning validation set (at tuning) and the ultimate hold-out test set, giving an equivalent measure of comparison for cross-experiment.

As the potential sensitivity of Root Mean Square Error (RMSE) to the absolute price level, a supporting measure named Mean Absolute Percentage Error (MAPE) is also used to supply a scale-independent perspective. MAPE estimates the error in proportion to the actual value and thus facilitates a relative performance comparison for different price ranges or over different data sets functioning in different units.

In addition to point estimates, every iteration saves detailed training and validation loss curves (loss and val_loss against epoch). A plot of the curves enlightens learning dynamics—like explosive convergence, stabilization, divergence, or indications of overfitting and underfitting—that may remain invisible behind numberic summariies alone. Combined with RMSE and MAPE, the learning curves provide quantitative and qualitative insight into how various architectural or hyper-parameter adjustments influence prediction quality and generalizability.

III. EXPERIMENTS

This section outlines the large experimental campaign. Each experiment is based on the canonical pipeline detailed in Section 2, trained for ten epochs with early stopping, and tested on the basis of RMSE, MAPE, and learning-curve diagnostics. One design dimension is changed at a time to enable the isolation of the effect of a specific factor.

1. Baseline Configuration

The baseline network is two stacked LSTM layers (5 units each) and one linear Dense neuron. The inputs are 60-day windows of the High price per day, normalized to [0,1]. The minimalist network provides a consistent but intentionally conservative baseline against which all the subsequent versions are compared.

2. Experiment #1: Other Data Preprocessing Methods
    2.1. Experiment 1.1: z-Score Standardization
    2.2. Experiment 1.2: Log Transformation

Two variants check whether using an alternative scale or form of data spreading accelerates learning. Sub-experiment 1.1 transforms the entire sequence of prices to z-scores (zero mean, unit variance) prior to applying Min–Max scaling, and sub-experiment 1.2 applies a natural-log transform to eliminate the largest outliers. Baseline comparison checks whether the LSTM is impacted by the spreading of input data.

3. Experiment #2: Adding Layers
    3.1. Experiment 2.1: Adding One Additional LSTM Layer
    3.2. Experiment 2.2: Adding Two Additional LSTM Layers
    3.3. Experiment 2.3: Adding One Additional Dense Layer

Depth is investigated by increasing the recurrent stack size. Sub-experiment 2.1 introduces a third LSTM; Sub-experiment 2.2 stacks four; Sub-experiment 2.3 maintains two LSTMs but introduces a 64-unit ReLU Dense bottleneck in between. These sub-experiments investigate whether increasing the number of levels of abstraction improves generalization or only overfits a small set.

4. Experiment #3: Changing RNN
    4.1. Experiment 3.1: Using SimpleRNN Layers
    4.2. Experiment 3.2: Using GRU Layers

To determine the significance of gates, replace the LSTMs with SimpleRNN cells without gates in sub-experiment 3.1 and GRU cells keeping gates but at a reduced footprint in sub-experiment 3.2. Performance differences here serve to signal the extent to which baseline accuracy is explained by LSTM-exclusive memory management.

5. Experiment #4: Changing Number of Units
    5.1. Experiment 4.1: Increasing the Number of Units
    5.2. Experiment 4.2: Decreasing the Number of Units
    5.3. Experiment 4.3: Varying Units in Different Layers

Model capacity is different in three ways: Sub-experiment 4.1 increases the hidden size of both LSTMs to 10 units each, sub-experiment 4.2 reduces them to 2 units, and sub-experiment 4.3 staggers capacity (10 units for the first layer, 5 for the second). These differences test the bias–variance trade-off and if asymmetric layer sizes are useful in learning low- and high-level temporal patterns.

6. Experiment #5: Changing Learning Rate
    6.1. Experiment 5.1: Increasing the Learning Rate
    6.2. Experiment 5.2: Decreasing the Learning Rate
    6.3. Experiment 5.3: Using a Learning Rate Scheduler

The fixed step size of RMSProp is altered to explore optimisation dynamics. Sub-experiment 5.1 uses a faster rate ($\eta = 0.01$), sub-experiment 5.2 a slower one ($\eta = 0.0001$), and sub-experiment 5.3 maintains $\eta = 0.001$ but combines it with a cosine-decay scheduler. Observing convergence rate and plateauing behaviour accounts for why learning-rate choice influences sensitivity to the task.

7. Experiment #6: Changing Sequence Length
    7.1. Experiment 6.1: Shorter Sequence Length
    7.2. Experiment 6.2: Longer Sequence Length
    7.3. Experiment 6.3: Very Short Sequence Length

Context length LLL is cut short to 20 days (sub-experiment 6.1), extended to 120 days (sub-experiment 6.2), and shortened to an ultra-short 10 days (sub-experiment 6.3). These

sub-experiments tell us whether longer memory is helpful to the network or whether irrelevant history just adds noise.

8. Experiment #7: Different Activation Functions
    - 8.1. Experiment 7.1: Using `relu` Activation
    - 8.2. Experiment 7.2: Using `sigmoid` Activation
    - 8.3. Experiment 7.3: `relu` in the Output Dense Layer
    - 8.4. Experiment 7.4: `sigmoid` in the Output Dense Layer

Activation surfaces in the recurrent layers and at the output are modified. Sub-experiments 7.1 substitutes the LSTM default tanh with ReLU, and sub-experiment 7.2 substitutes the LSTM default sigmoid. Sub-experiment 7.3 modifies the Dense output from linear to ReLU, and sub-experiment 7.4 modifies it to sigmoid. These combinations check gradient flow and range-restriction effects in both hidden and output spaces.

9. Experiment #8: Adding Dropout to Input or LSTM Layers
    - 9.1. Experiment 8.1: Adding Dropout After the First LSTM Layer
    - 9.2. Experiment 8.2: Adding Dropout After the Second LSTM Layer
    - 9.3. Experiment 8.3: Adding Dropout to Both LSTM Layers
    - 9.4. Experiment 8.4: Adding Dropout to the Input Sequence
    - 9.5. Experiment 8.5: Combining Dropout in LSTM Layers and Input

Regularisation is included in different locations. Sub-experiment 8.1 includes dropout ($p = 0.2$) following the first LSTM, sub-experiment 8.2 following the second, and sub-experiment 8.3 following both. Sub-experiment 8.4 includes the same dropout value for the input sequence of 60 days, and sub-experiment 8.5 includes input dropout and layer-wise dropout. Comparison of validation curves tells us where stochastic deactivation is most effective.

10. Experiment #9: Different Optimizers
    - 10.1. Experiment 9.1: Using Adam Optimizer
    - 10.2. Experiment 9.2: Using SGD Optimizer
    - 10.3. Experiment 9.3: Using Adagrad Optimizer

The baseline RMSProp is substituted by three widely used alternatives. Sub-experiment 9.1 employs Adam, sub-experiment 9.2 employs SGD with momentum 0.9, and sub-experiment 9.3 employs Adagrad. This group examines the impact of optimiser choice on convergence rate and final error for sequential price data.

11. Experiment #10: Lagged Features (Feature Engineering)
    - 11.1. Experiment 10.1: Adding the Previous Day's Price
    - 11.2. Experiment 10.2: Adding a Moving Average
    - 11.3. Experiment 10.3: Adding the Price Change
    - 11.4. Experiment 10.4: Combining Multiple Lagged Features

The single-channel input is complemented by specially crafted signals. Sub-experiment 10.1 features the previous day's price (lag-1), sub-experiment 10.2 features a 5-day moving average, and sub-experiment 10.3 features the day-to-day price change Δp_t. Sub-experiment 10.4 features all three attributes, yielding a three-channel tensor at each time step. These tests determine whether an overt statistical context allows the LSTM to identify trends that otherwise it would miss.

IV. Results and Analysis

This section discusses the results and analysis of each experiment that will explain how the experimented hyperparameters and RNN architectures affect the model's overall performance. It is noteworthy to showcase first the settings and the performance of the baseline model

that was used in each experiment before moving on to the experiments so we can perform a comparative analysis on each experiment.

The following settings are the hyperparameters and architecture of the baseline model:

TABLE 1
BASELINE MODEL CONFIGURATION

DATA PREPROCESSING METHOD	MIN-MAX SCALER
SEQUENCE LENGTH	60
RNN MODEL	LONG SHORT-TERM MEMORY
NUMBER OF RECURRENT LAYERS	2
NUMBER OF UNITS	5
ACTIVATION FUNCTION	LINEAR
OPTIMIZER	ROOT MEAN SQUARE PROPAGATION
LEARNING RATE	0.001
DROPOUT RATE	NONE

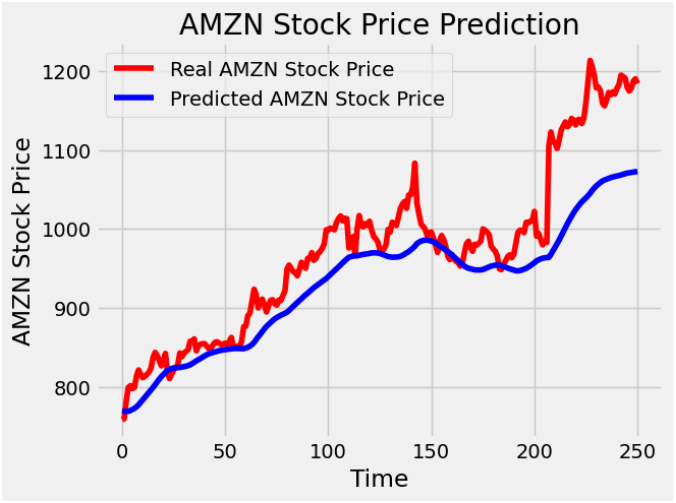

Fig. 1. Plot Prediction of the Baseline Model.

The baseline model achieved an RMSE of 59.76 which is pretty impressive for a baseline model. The default configuration already showed decent prediction capability but still struggled with overfitting at certain data points, especially when it tried to predict sudden bumps in stock prices.

A. Experiment #1: Other Preprocessing Methods

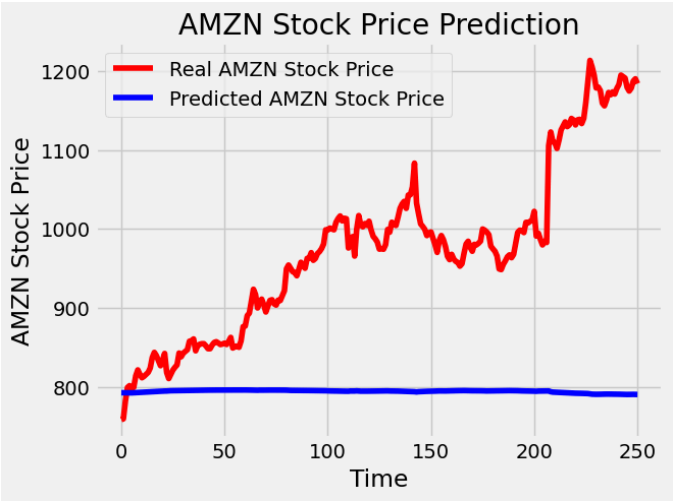

Fig. 2. Plot Prediction of the Sub-Experiment 1.1: z-Score Standardization.

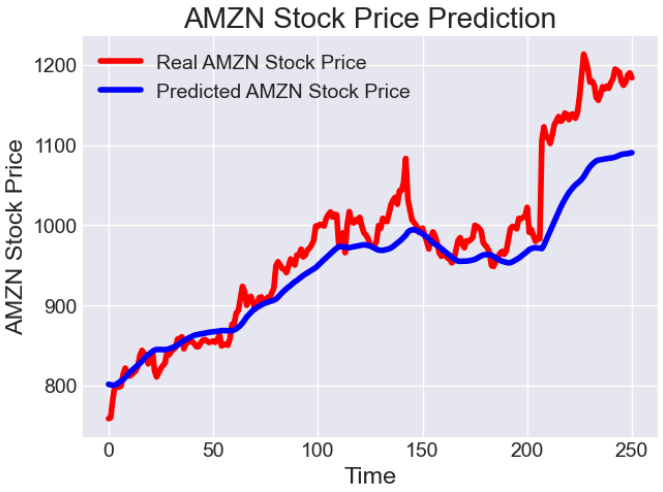

Fig. 3. Plot Prediction of the Sub-Experiment 1.2: Log Transformation.

TABLE 2
EXPERIMENT #1 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	**RMSE**	**LOSS**
MIN-MAX SCALER	59.76	3.3517E-04
Z-SCORE STANDARDIZATION	210.39	0.0036
LOG TRANSFORMATION	50.76	3.5402E-04

z-Score Standardization and Log Transformation were applied for the first experiment. As evident on Figures 2-3, Log Transformation showed the best predicting capability against z-Score Standardization with RMSE of 50.76 and 210.39 respectively. Log Transformation also performed better than the Min-Max Scaler of the baseline

model. This result indicates that the distribution of the stock price data might have characteristics like skewness or non-constant variance that were better handled by the Log Transformation. Although the Min-Max Scaler was almost at par with the performance of the model on Log Transformation, as mentioned above, there are characteristics of the stock price data that might have favored the properties of the logarithmic scale. On the other hand, the poor performance of z-Score Standardization suggests that the scaling of the data to have zero mean and unit variance might have obscured important patterns or introduced difficulties for the model to learn the temporal dependencies in the stock prices.

While the z-Score Standardization resulted in a much higher training loss, both Min-Max Scaler and Log Transformation led to very low training losses. However, when we look at the RMSE, Log Transformation performed significantly better than Min-Max Scaler (50.76 vs. 59.76). This highlights a critical point that a good fit on the training data doesn't always translate to good generalization on unseen data. The Log Transformation might have created a training landscape that allowed the model to learn more robust features that generalized better, even if the training loss wasn't drastically lower than Min-Max.

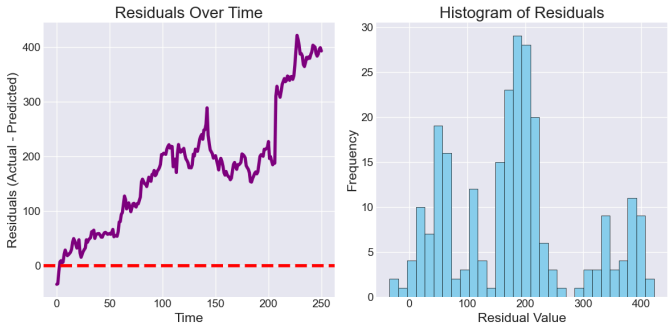

Fig. 4. Residual Plot of the Sub-Experiment 1.1: z-Score Standardization.

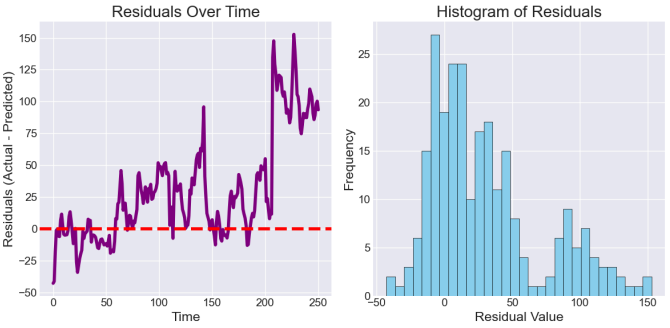

Fig. 5. Residual Plot of the Sub-Experiment 1.2: Log Transformation.

It is also important to check the residual plot of both z-Score Standardization and Log Transformation as seen on Figures 4-5.

The distribution of errors on z-Score Standardization are far from the center (which is zero). In this plot, the distribution seems skewed to the right, towards positive residual values. This reinforces the observation from the time series plot that the model tends to underpredict more often. The histogram in Figure 4 also shows a wide spread of residual values, ranging from below zero to over 400. Its shape does not resemble a normal (bell-shaped) distribution. It appears to have multiple peaks and is quite irregular, further suggesting that the errors are not random noise but have some underlying structure.

Comparing with Log Transformation in Figure 5, the residuals over time is near zero although bumps at around time 200 yet have smaller residuals. The Histogram also has showcased the error distribution around zero which almost resembles a normal distribution. These findings of residuals aligned with the RMSE for both z-Score Standardization and Log Transformation.

B. *Experiment #2: Adding Layers*

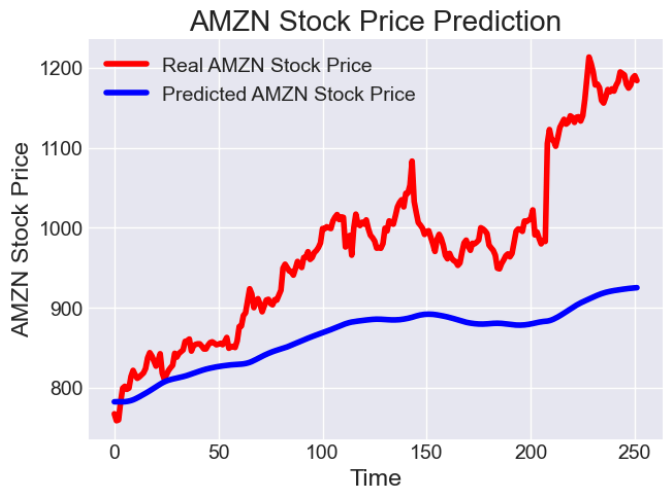

Fig. 6. Plot Prediction of the Sub-Experiment 2.1: Adding One Additional LSTM Layer.

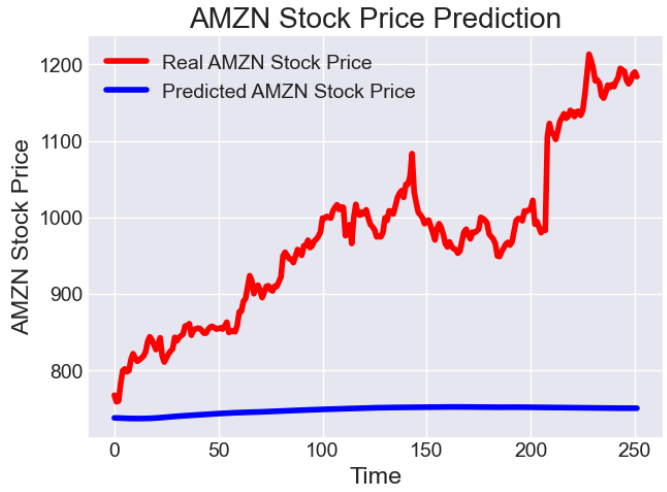

Fig. 7. Plot Prediction of the Sub-Experiment 2.2: Adding Two Additional LSTM Layers.

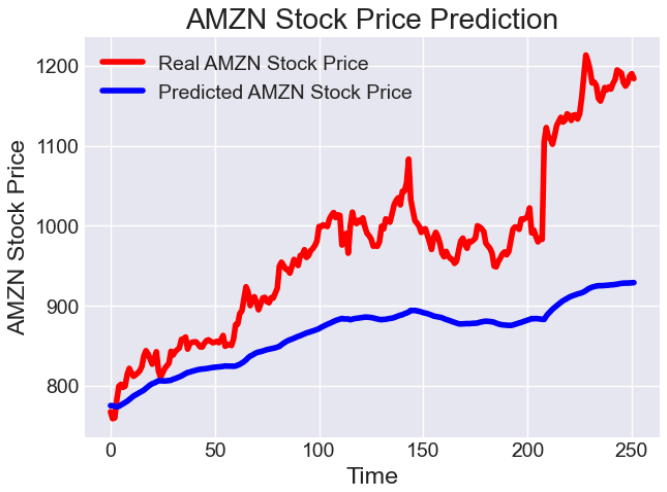

Fig. 8. Plot Prediction of the Sub-Experiment 2.3: Adding One Additional Dense Layer.

TABLE 3
EXPERIMENT #2 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	**RMSE**	**LOSS**	**VALIDATION LOSS**
TWO LSTM LAYERS	59.76	3.3517E-04	N/A
THREE LSTM LAYERS	132.79	4.6127E-04	0.0011
FOUR LSTM LAYERS	249.10	9.0589E-04	0.0047
TWO DENSE LAYERS	130.41	2.8761E-04	7.5200E-04

The results of Experiment #2 indicates a trend of diminishing and then negative returns with increasing model complexity.

Adding one additional LSTM layer (three total) led to a significant increase in RMSE (132.79) compared to the baseline two LSTM layers (59.76). This suggests that increasing the depth of the recurrent part of the network beyond two layers did not improve the model's ability to generalize to the test data. The higher validation loss also supports this, indicating potential overfitting or difficulty in training a deeper recurrent network effectively for this dataset.

Further increasing the LSTM layers to four resulted in an even more substantial increase in RMSE (249.10). This reinforces the idea that a deeper LSTM network is detrimental to the model's performance. The increasing validation loss further suggests instability or severe overfitting.

Interestingly, adding two dense layers after an initial LSTM layer also resulted in a higher RMSE (130.41) than the baseline two LSTM layers. While the training loss was the lowest among the variations, the higher validation loss and test RMSE indicate that these dense layers, in this configuration, did not help the model capture the temporal dependencies in the stock price data and might have introduced overfitting to the training set.

Adding more dense layers led to a significant degradation in the model's ability to predict unseen data, likely due to increased complexity without a corresponding increase in the model's ability to learn the underlying patterns or due to overfitting. This highlights the importance of finding the right balance in model complexity.

C. Experiment #3: Changing RNN

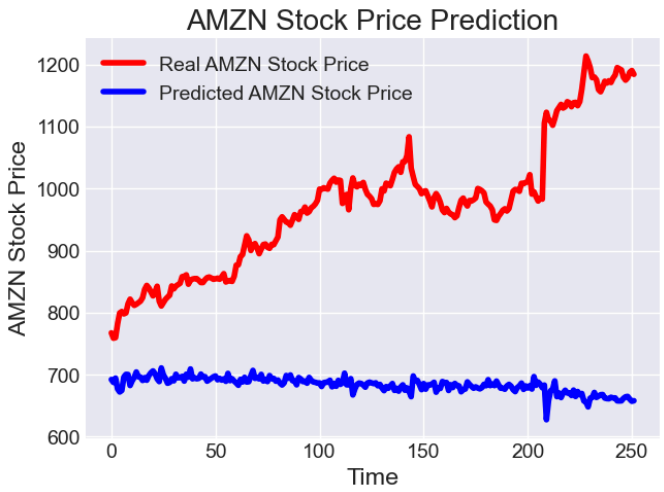

Fig. 9. Plot Prediction of the Sub-Experiment 3.1: Using SimpleRNN Layers.

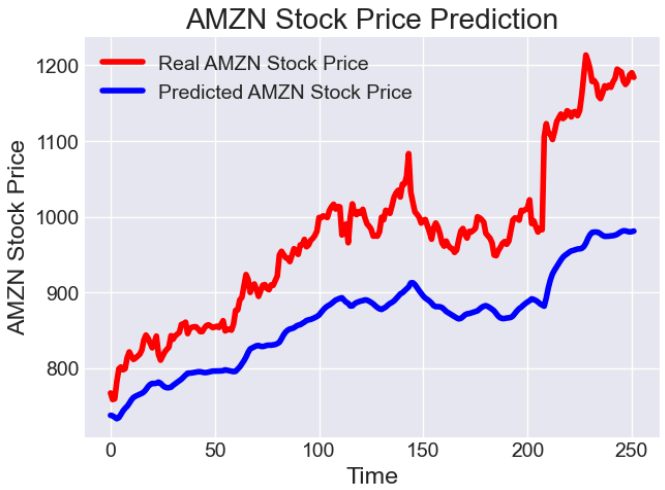

Fig. 10. Plot Prediction of the Sub-Experiment 3.2: Using GRU Layers.

TABLE 4
EXPERIMENT #3 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	RMSE	LOSS	VALIDATION LOSS
LONG SHORT-TERM MEMORY	59.76	3.3517E-04	N/A
SIMPLERNN	313.35	4.6371E-04	0.0144
GATED RECURRENT UNIT	119.79	1.9683E-04	0.0031

The choice of RNN architecture significantly impacts the model's performance. For this stock price prediction problem, the LSTM model proved to be the most suitable, outperforming both SimpleRNN and GRU.

SimpleRNN performed significantly worse, exhibiting a very high RMSE of 313.35. While the training loss was in the same order of magnitude as LSTM, the drastically higher validation loss indicates a severe issue with generalization, likely due to the well-known vanishing/exploding gradient problems in vanilla RNNs, making it difficult to learn long-range dependencies in the time series data. The prediction plot visually confirms this as seen on Figure 9, showing a predicted line that barely follows the trend of the actual stock price.

On the other hand, GRU showed an improvement over SimpleRNN, achieving an RMSE of 119.79. Its lower training and validation loss compared to SimpleRNN suggests that the gating mechanism in GRU helped it capture the temporal dynamics more

effectively. However, the performance was still considerably worse than the baseline LSTM. The prediction plot for GRU in Figure 10 shows a better attempt at following the stock price movements than SimpleRNN, but it still lags behind and misses many of the bumps in the dataset.

Overall, this experiment highlights the importance of using more advanced recurrent units like LSTMs when dealing with sequential data where long-term dependencies might be crucial.

D. Experiment #4: Changing Number of Units

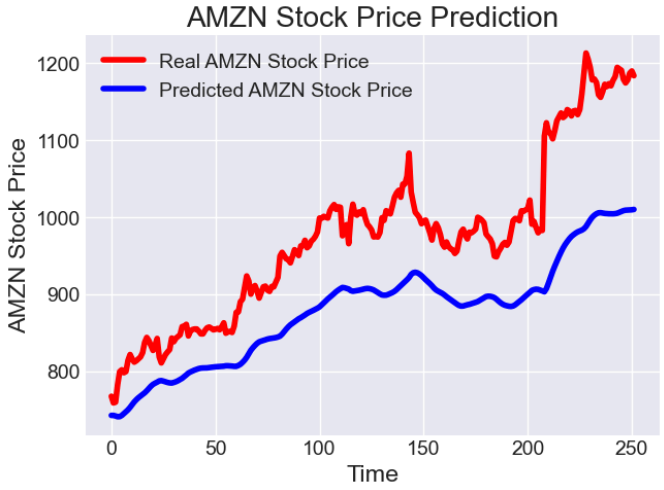

Fig. 11. Plot Prediction of the Sub-Experiment 4.1: Increasing the Number of Units.

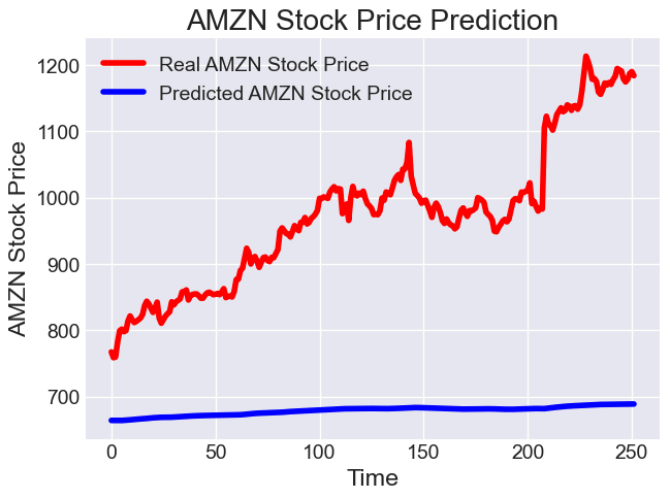

Fig. 12. Plot Prediction of the Sub-Experiment 4.2: Decreasing the Number of Units.

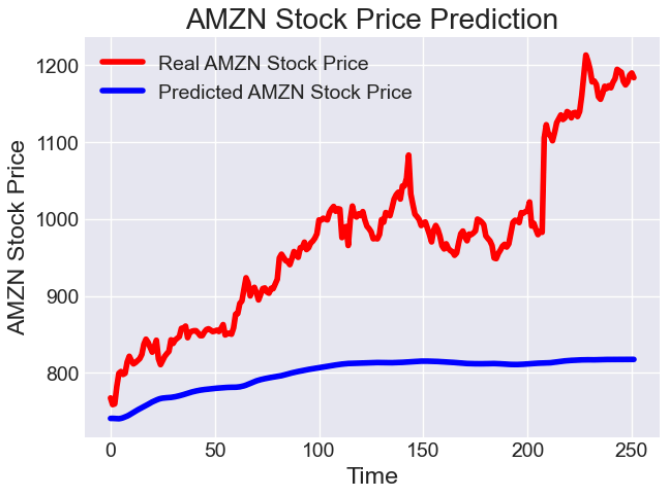

Fig. 13. Plot Prediction of the Sub-Experiment 4.3: Varying Units in Different Layers.

TABLE 5
EXPERIMENT #4 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	RMSE	LOSS	VALIDATION LOSS
5 UNITS (ALL LSTM LAYERS)	59.76	3.3517E-04	N/A
20 UNITS (ALL LSTM LAYERS)	104.97	4.1070E-04	0.0029
2 UNITS (ALL LSTM LAYERS)	312.27	6.9705E-04	0.0235
10 UNITS (FIRST LSTM LAYER) 3 UNITS (SECOND LSTM LAYER)	197.03	2.8610E-04	0.0033

The baseline model with 5 units in each of the two LSTM layers appeared to be the most effective among the tested variations for this experiment. Deviating from this number, either by increasing, decreasing, or varying the units across layers, led to a degradation in the model's performance on the unseen test data.

Increasing the number of units to 20 in both LSTM layers resulted in a higher RMSE (104.97) compared to the baseline. While the training loss increased slightly and a validation loss was observed (suggesting better monitoring of generalization), the higher RMSE on the testing set indicates that more units did not translate to better predictive accuracy, potentially leading to overfitting or learning more noise. Figure 11 shows a model that seems to follow the general trend but with less accuracy in capturing the finer bumps.

Decreasing the number of units drastically to 2 in both LSTM layers led to a substantial increase in RMSE (312.27). The significantly higher loss and validation loss suggest that with too few units, the model lacks the capacity to learn the complex patterns in the stock price data, resulting in very poor predictions, as evident in Figure 12 where the predicted line is very smooth and far from the actual price.

Varying the number of units between layers (10 in the first, 3 in the second) also resulted in a higher RMSE (197.03) than the baseline. This configuration did not offer any advantage, suggesting that a consistent number of units across layers might be more suitable or that a more systematic search for the optimal unit configuration is needed. Figure 13 shows some attempt to follow the trend but with significant deviations.

E. Experiment #5: Changing Learning Rate

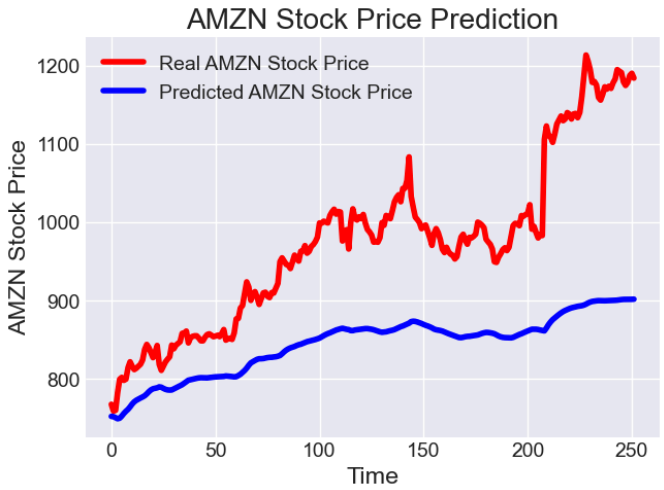

Fig. 14. Plot Prediction of the Sub-Experiment 5.1: Increasing the Learning Rate.

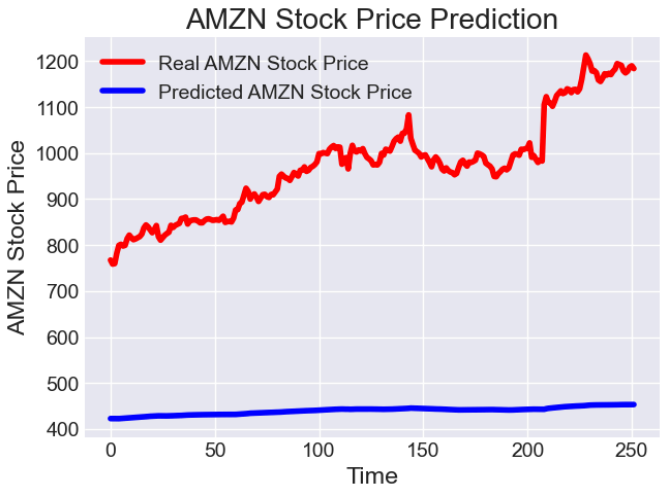

Fig. 15. Plot Prediction of the Sub-Experiment 5.2: Decreasing the Learning Rate.

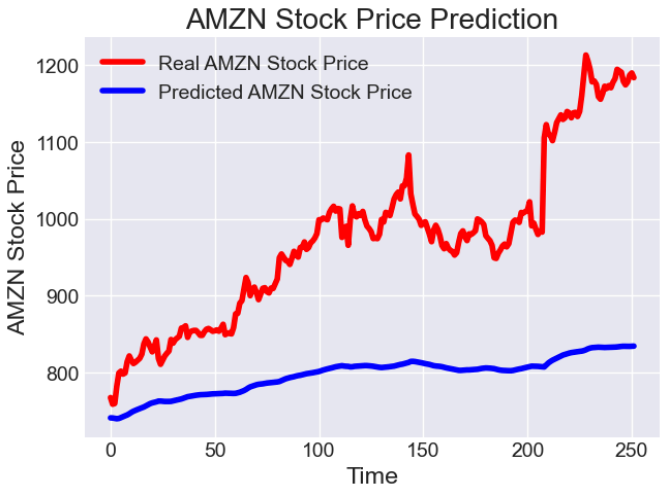

Fig. 16. Plot Prediction of the Sub-Experiment 5.3: Using a Learning Rate Scheduler.

Fig. 17. Training/Validation Loss Plot of the Sub-Experiment 5.1: Increasing the Learning Rate.

Fig. 18. Training/Validation Loss Plot of the Sub-Experiment 5.2: Decreasing the Learning Rate.

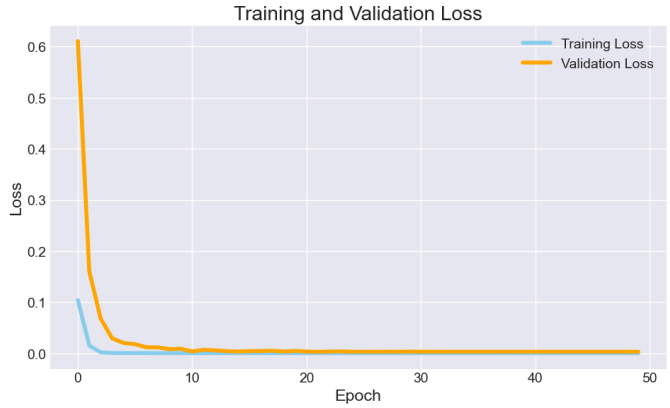

Fig. 19. Training/Validation Loss Plot of the Sub-Experiment 5.3: Using a Learning Rate Scheduler.

TABLE 6
EXPERIMENT #5 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	RMSE	LOSS	VALIDATION LOSS
LEARNING RATE = 0.001	59.76	3.3517E-04	N/A
LEARNING RATE = 0.01	149.24	6.4671E-04	0.0015
LEARNING RATE = 0.001	543.50	0.0107	0.2470
REDUCELRONPLATEAU	196.73	0.0030	1.0000E-05

For this experiment, it's noteworthy to include the training and validation loss plot due to the interesting behavior of the model with respect to different fixed learning rates and its impact to the loss function. The baseline learning rate of 0.001 appeared to be the most suitable among the tested fixed learning rates for this model and dataset.

Increasing the learning rate to 0.01 resulted in a significantly higher RMSE (149.24). Figure 17 shows rapid fluctuations and potential instability, suggesting that the model might have been overshooting the optimal weights during training, leading to poor generalization. Figure 14 shows a model that struggles to follow the actual price movements.

Decreasing the learning rate drastically to 0.0001 led to a dramatically higher RMSE (543.50). Figure 18 shows a very slow decrease in loss, indicating that the model was learning extremely slowly and likely didn't converge to a good solution within the training epochs. Figure 15 shows a nearly flat predicted line, far from the actual stock price.

Using the ReduceLROnPlateau learning rate scheduler also resulted in a higher RMSE (196.73) compared to the baseline. While the validation loss decreased significantly and became very small towards the end of training (as seen in Figure 19), this didn't translate to better performance on the test set. It's possible that the scheduler reacted to plateaus in the validation loss in a way that didn't optimize for the best generalization, suggesting the importance of careful tuning of scheduler parameters. Figure 16 shows a model that follows the general trend but with significant deviations. This experiment underscores the sensitivity of neural network training to the learning rate.

F. *Experiment #6: Changing Sequence Length*

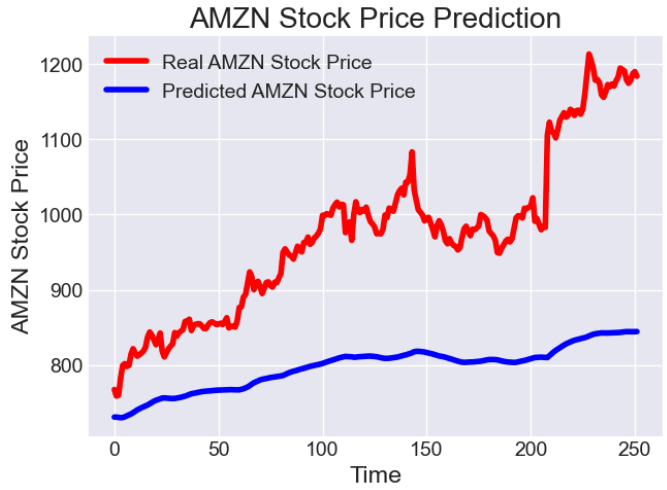

Fig. 20. Plot Prediction of the Sub-Experiment 6.1: Shorter Sequence Length

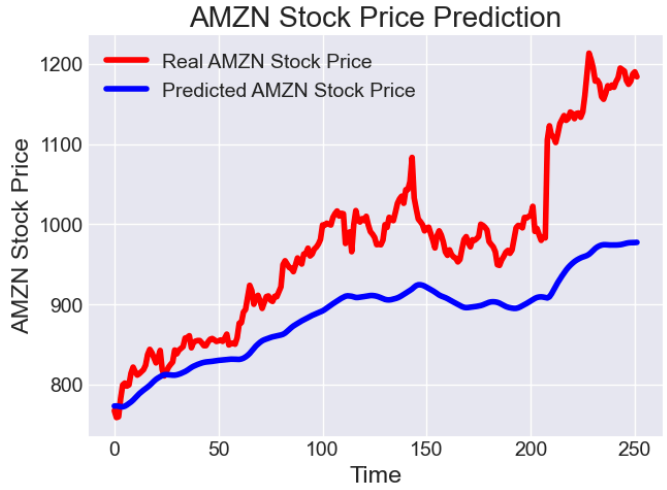

Fig. 21. Plot Prediction of the Sub-Experiment 6.2: Longer Sequence Length

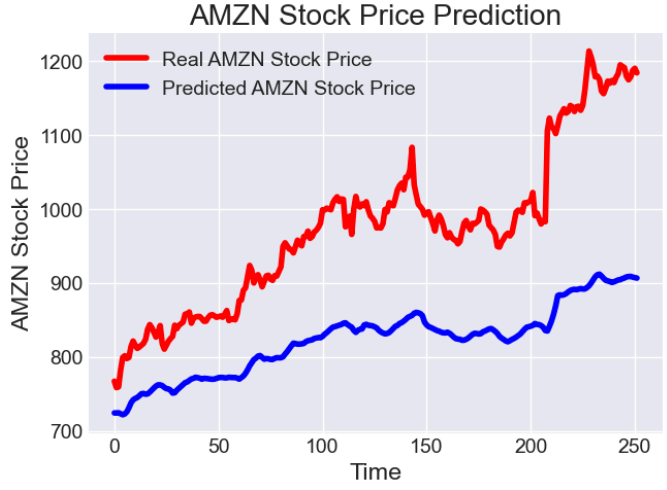

Fig. 22. Plot Prediction of the Sub-Experiment 6.3: Very Short Sequence Length

TABLE 7
EXPERIMENT #6 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	RMSE	LOSS	VALIDATION LOSS
SEQUENCE LENGTH = 60	59.76	3.3517E-04	N/A
SEQUENCE LENGTH = 20	194.53	3.1755E-04	0.0042
SEQUENCE LENGTH = 120	106.63	3.3790E-04	8.2210E-04
SEQUENCE LENGTH = 5	163.52	2.0697E-04	0.0042

The results of this experiment indicate that the choice of sequence length significantly impacts the model's ability to predict stock prices. Deviating from the baseline sequence length of 60 generally led to worse performance.

A shorter sequence length of 20 resulted in a much higher RMSE (194.53). Figure 20 shows a model that reacts more quickly to short-term fluctuations but fails to capture the broader trends effectively. The higher validation loss also suggests that training on shorter sequences might lead to a less robust model that doesn't generalize well to longer-term patterns.

A longer sequence length of 120 also led to a higher RMSE (106.63) compared to the baseline, although the degradation was less severe than with the shorter sequence. Figure 21 shows a smoother predicted line, potentially capturing longer trends but missing some of the shorter-term changes. The slightly higher validation loss might indicate challenges in training on very long sequences.

Lastly, a very short sequence length of 5 resulted in a substantial increase in RMSE (163.52). The prediction plot shows a highly erratic predicted line that seems to be mostly noise and fails to capture any meaningful trends. The higher validation loss again suggests poor generalization due to the model not having enough historical context to learn from.

This experiment highlights the importance of selecting an appropriate sequence length that balances the need to capture both short-term and long-term patterns in the time series data.

G. *Experiment #7: Different Activation Functions*

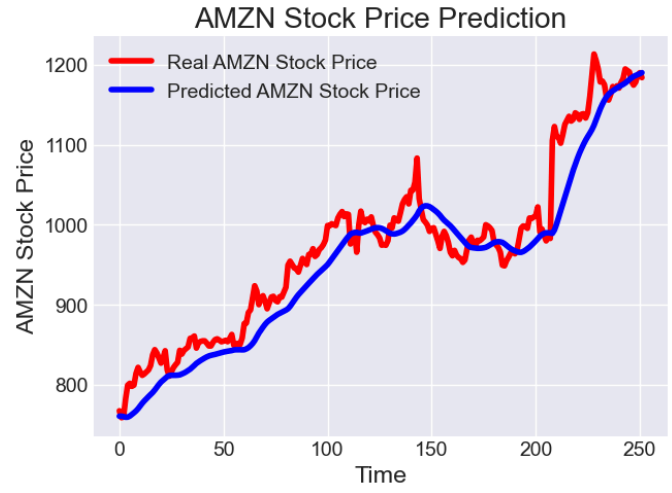

Fig. 23. Plot Prediction of the Sub-Experiment 7.1: Using `relu` Activation

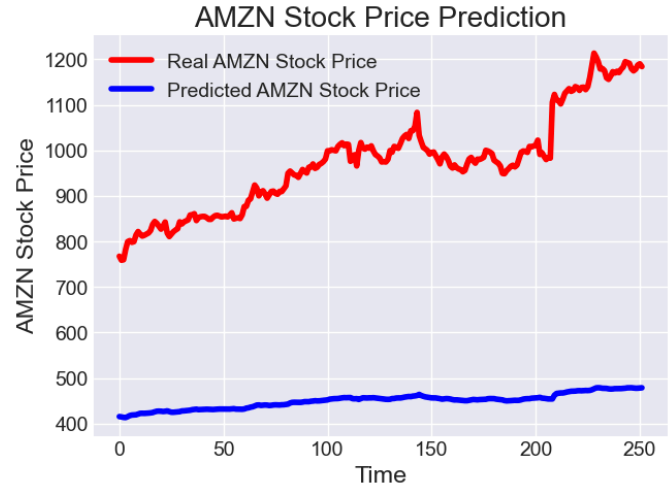

Fig. 24. Plot Prediction of the Sub-Experiment 7.2: Using `sigmoid` Activation

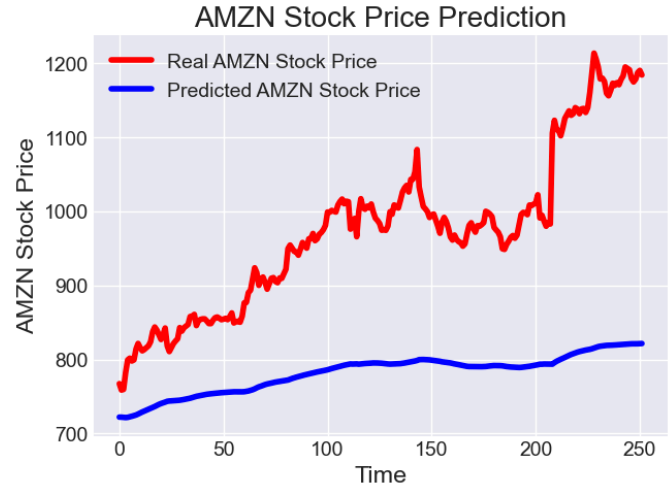

Fig. 25. Plot Prediction of the Sub-Experiment 7.3: `relu` in the Output Dense Layer

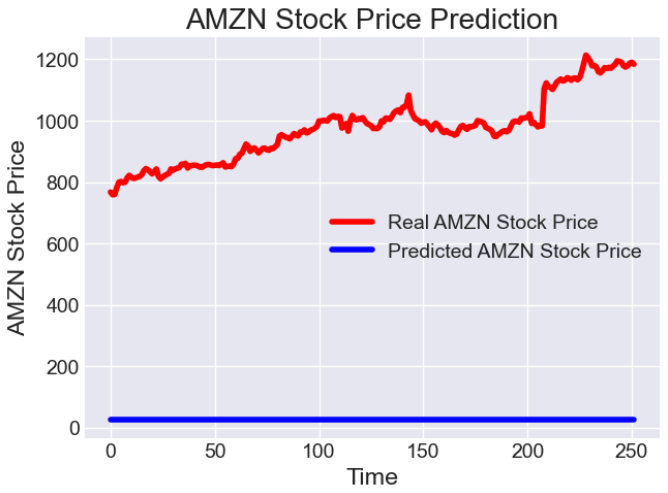

Fig. 26. Plot Prediction of the Sub-Experiment 7.4: `sigmoid` in the Output Dense Layer

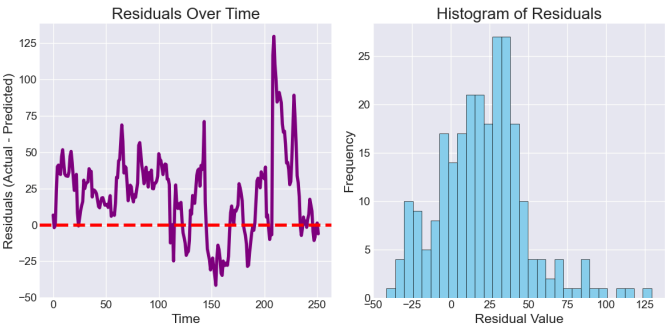

Fig. 27. Residual Plot of the Sub-Experiment 7.1: Using `relu` Activation

TABLE 8
EXPERIMENT #7 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	RMSE	LOSS	VALIDATION LOSS
LINEAR	59.76	3.3517E-04	N/A
RELU	34.76	5.2260E-04	0.0019
SIGMOID	532.26	0.0216	0.2607
RELU IN THE OUTPUT DENSE LAYER	210.50	4.1720E-04	0.0061
SIGMOID IN THE OUTPUT DENSE LAYER	953.58	0.1079	1.1623

The choice of activation function significantly impacted the model's performance in this experiment. It is also noteworthy to include the residual plot of sub-experiment 7.1 which is using the relu activation function as seen on Figure 27.

Speaking of, using ReLu activation in the LSTM layers resulted in a substantial improvement, achieving the lowest RMSE of 34.76. Figure 23 shows a model that follows the actual stock price more closely than the baseline. Figure 27 shows residuals that are generally smaller and more centered around zero compared to what we've seen in some other experiments, although there's still some structure and non-normality. The validation loss also indicates reasonable generalization. ReLU's ability to introduce non-linearity without saturating for positive inputs likely helped the LSTM layers learn complex patterns more effectively.

Using sigmoid activation in the LSTM layers led to a dramatic decrease in performance, with a very high RMSE of 532.26. Figure 24 shows a predicted line that is almost flat and completely fails to capture the stock price movements. The very high loss and validation loss indicate severe difficulties in training the model with sigmoid as the primary activation in the recurrent layers, likely due to the vanishing gradient problem being exacerbated over long sequences.

Using relu only in the output dense layer resulted in a significantly higher RMSE (210.50) compared to using ReLU in the LSTM layers. Figure 25 shows a model that follows the general trend but with large deviations. This suggests that the activation function in the recurrent layers plays a more critical role in learning the temporal dependencies for this task.

Using sigmoid only in the output dense layer led to the worst performance, with an extremely high RMSE of 953.58. Figure 26 shows a predicted line that is flat and near zero. The very high loss and validation loss indicate a complete failure of the model to learn anything meaningful. The sigmoid function, with its output range of 0 to 1, is likely not suitable as the final activation for predicting stock prices which have a much wider range.

H. Experiment #8: Adding Dropout to Input or LSTM Layers

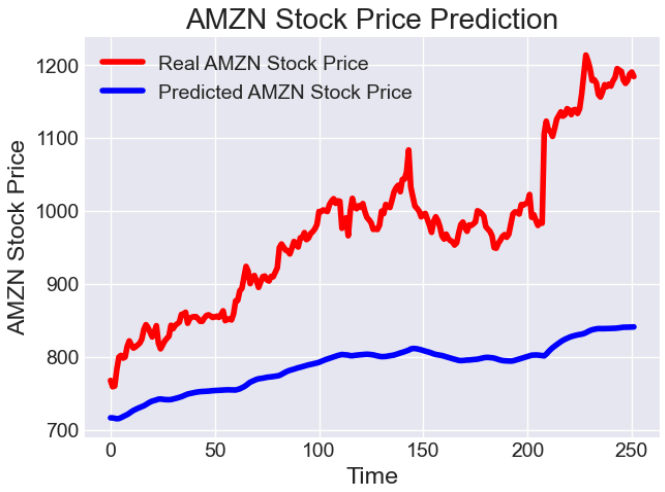

Fig. 28. Plot Prediction of the Sub-Experiment 8.1: Adding Dropout After the First LSTM Layer

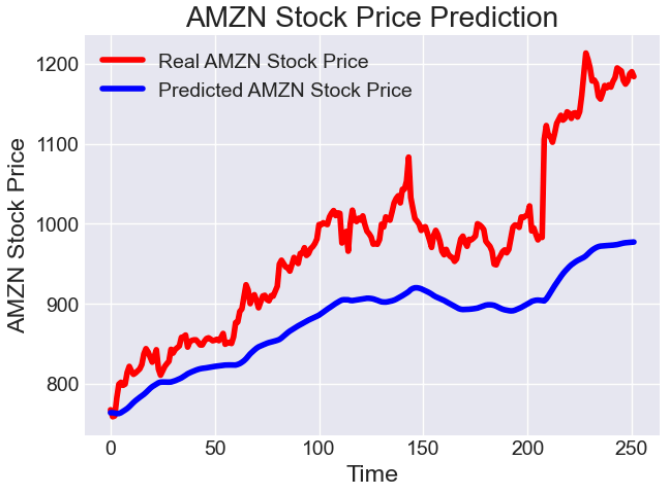

Fig. 29. Plot Prediction of the Sub-Experiment 8.2: Adding Dropout After the Second LSTM Layer

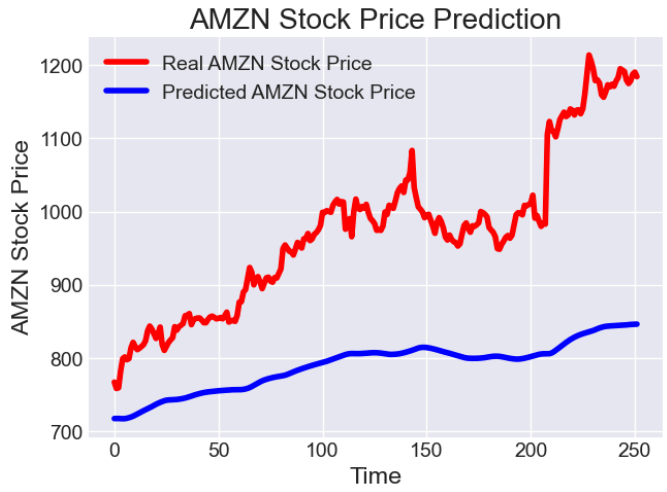

Fig. 30. Plot Prediction of the Sub-Experiment 8.3: Adding Dropout to Both LSTM Layers

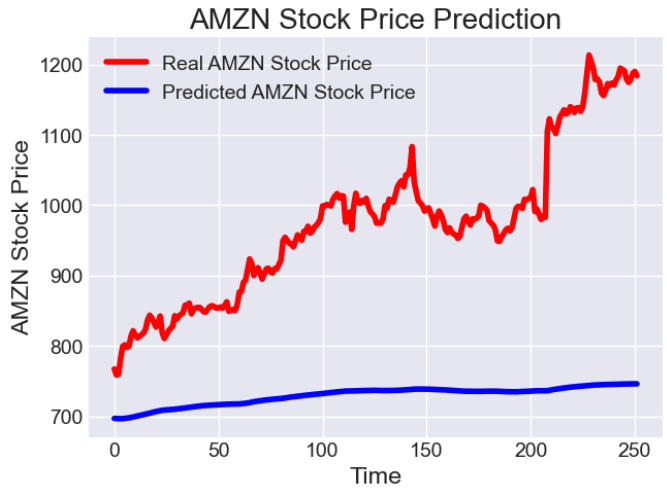

Fig. 31. Plot Prediction of the Sub-Experiment 8.4: Adding Dropout to the Input Sequence

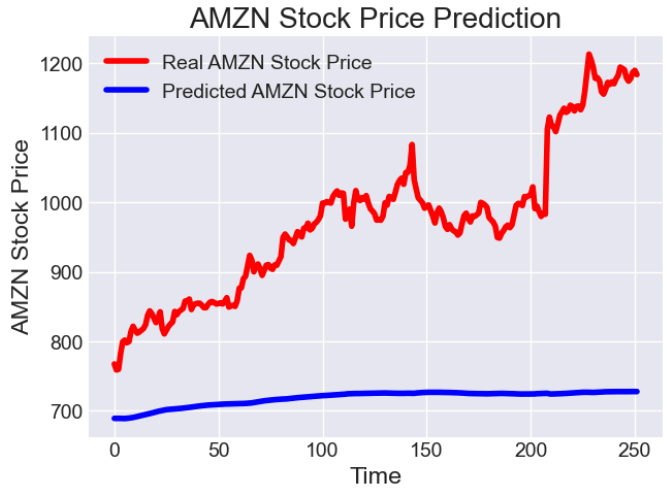

Fig. 32. Plot Prediction of the Sub-Experiment 8.5: Combining Dropout in LSTM Layers and Input

TABLE 9
EXPERIMENT #8 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	RMSE	LOSS	VALIDATION LOSS
NO DROPOUT	59.76	3.3517E-04	N/A
ADDING DROPOUT AFTER THE FIRST LSTM LAYER	202.19	6.8263E-04	0.0074
ADDING DROPOUT AFTER THE SECOND LSTM LAYER	110.30	0.0023	0.0011
ADDING DROPOUT TO BOTH LSTM LAYERS	199.52	0.0031	0.0073
ADDING DROPOUT TO THE INPUT SEQUENCE	262.84	0.0015	0.0126
COMBINING DROPOUT IN LSTM LAYERS AND INPUT	274.34	0.0041	0.0163

For this experiment, adding dropout with a rate of 0.2 generally did not improve generalization and instead led to a decrease in predictive accuracy.

Adding dropout after the first LSTM layer resulted in a significantly higher RMSE (202.19). Figure 28 shows a model that deviates considerably from the actual stock price. The higher validation loss suggests that this configuration might have hindered the model's ability to learn relevant features.

Adding dropout after the second LSTM layer also increased the RMSE (110.30), although the degradation was less severe than adding it after the first layer. Figure 29 shows a model that follows the general trend but with noticeable errors. The lower validation loss compared to the previous case might indicate better regularization in this specific placement.

Applying dropout to both LSTM layers resulted in a similarly high RMSE (199.52) as applying it only after the first layer. This suggests that indiscriminately adding dropout might over-regularize the network, preventing it from learning complex patterns.

Adding dropout to the input sequence led to the worst performance among the dropout experiments, with a very high RMSE of 262.84. Figure 31 shows a model that struggles significantly to follow the actual price. Applying dropout directly to the input might have removed crucial temporal information needed for prediction.

Combining dropout in both LSTM layers and the input sequence also resulted in very poor performance, with the highest RMSE of 274.34. This further reinforces the idea that excessive dropout can be detrimental to learning in this time series forecasting task.

Applying dropout after the second LSTM layer showed the least degradation, but still performed worse than the baseline. This suggests that for this dataset and baseline model architecture, overfitting might not have been a significant issue, or that the chosen dropout rate and placement were not optimal for regularization. It's possible that a lower dropout rate or a different placement strategy might yield better results, but with the current settings, dropout was not beneficial.

I. Experiment #9: Different Optimizers

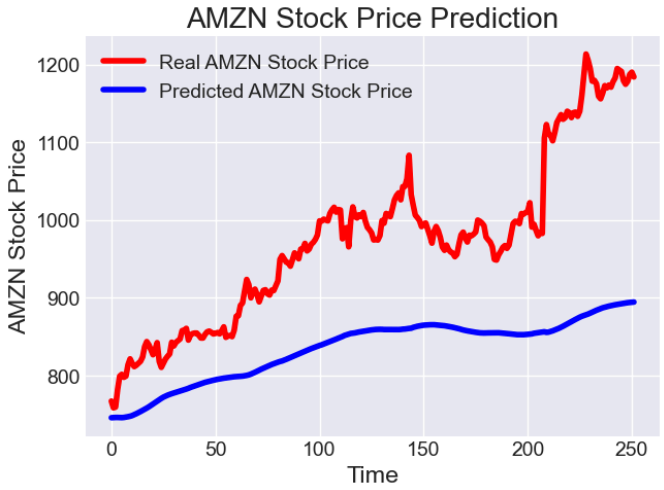

Fig. 33. Plot Prediction of the Sub-Experiment 9.1: Using Adam Optimizer

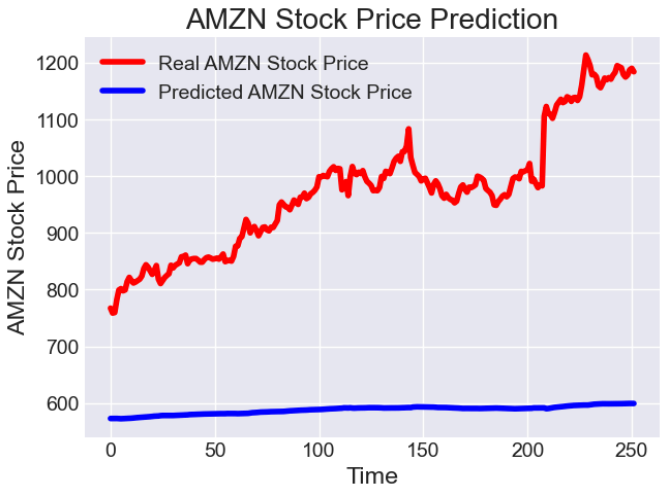

Fig. 34. Plot Prediction of the Sub-Experiment 9.2: Using SGD Optimizer

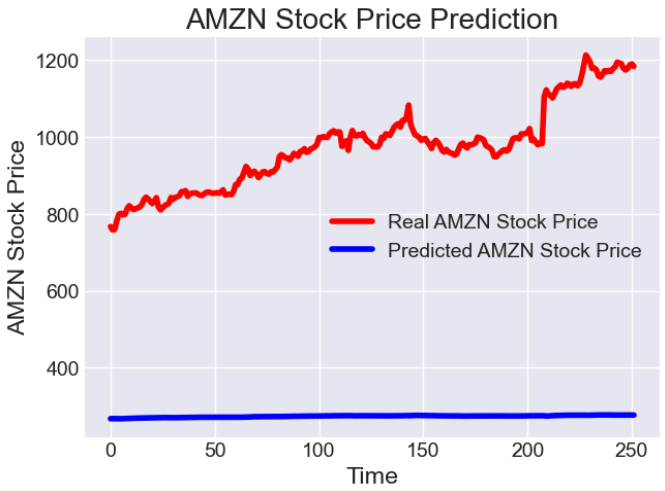

Fig. 35. Plot Prediction of the Sub-Experiment 9.3: Using Adagrad Optimizer

TABLE 10
EXPERIMENT #9 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	RMSE	LOSS	VALIDATION LOSS
RMSPROP OPTIMIZER	59.76	3.3517E-04	N/A
ADAM OPTIMIZER	157.99	3.7248E-04	0.0034
SGD OPTIMIZER	398.88	0.0011	0.0781
ADAGRAD OPTIMIZER	707.61	0.0301	0.5219

The choice of optimizer significantly impacted the model's performance in this experiment. The baseline RMSprop optimizer outperformed the other tested optimizers in this dataset.

Using the Adam optimizer resulted in a considerably higher RMSE (157.99) compared to RMSprop. While the training loss was in a similar range, the higher validation loss suggests that Adam might have led to a model that didn't generalize as well as RMSprop, potentially getting stuck in a local minimum or exhibiting different convergence behavior. Figure 33 shows a model that follows the general direction but with significant deviations and a tendency to underpredict.

The SGD optimizer performed much worse, with a very high RMSE of 398.88. Figure 34 shows a predicted line that is almost flat, indicating that the model struggled to learn the temporal dynamics of the stock price. The high validation loss further suggests instability or a very slow learning process with the given learning rate.

The Adagrad optimizer yielded the poorest performance, with an extremely high RMSE of 707.61. Figure 35 shows a predicted line that is completely flat and near zero. The very high training and validation loss indicate that Adagrad, with its adaptive learning rate based on past gradients, was not suitable for this dataset, possibly leading to a learning rate that decayed too quickly or inappropriately for the features in the data.

This highlights the importance of selecting an appropriate optimization algorithm that can effectively navigate the loss landscape for the given data and model architecture. The adaptive learning rate optimizers (Adam and Adagrad) and the simpler gradient descent (SGD) did not converge to a model with comparable predictive accuracy to RMSprop.

J. Experiment #10: Lagged Features (Feature Engineering)

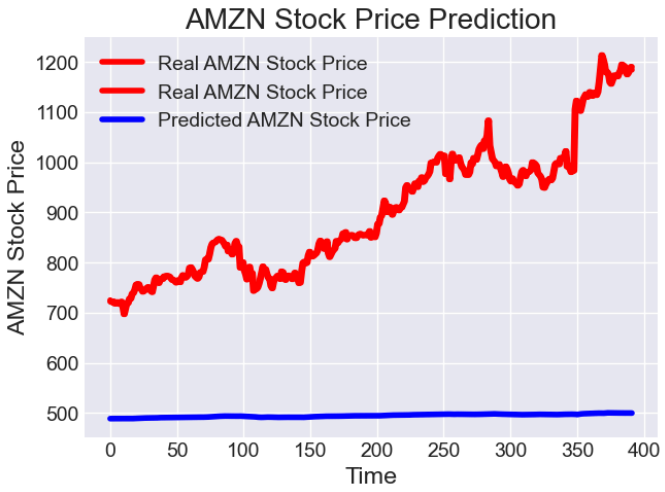

Fig. 36. Plot Prediction of the Sub-Experiment 10.1: Adding the Previous Day's Price

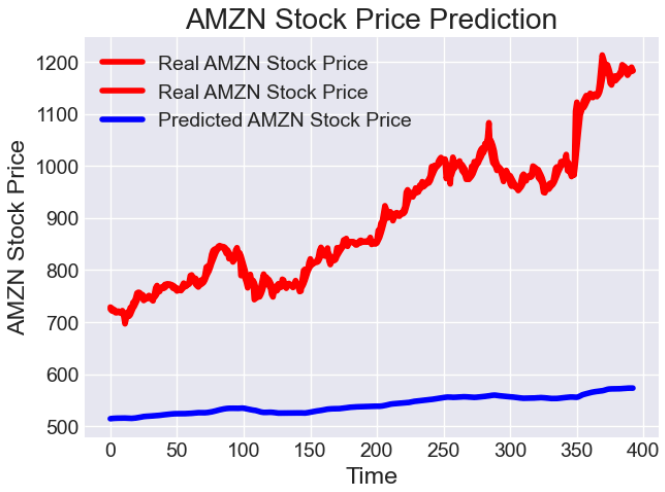

Fig. 37. Plot Prediction of the Sub-Experiment 10.2: Adding a Moving Average

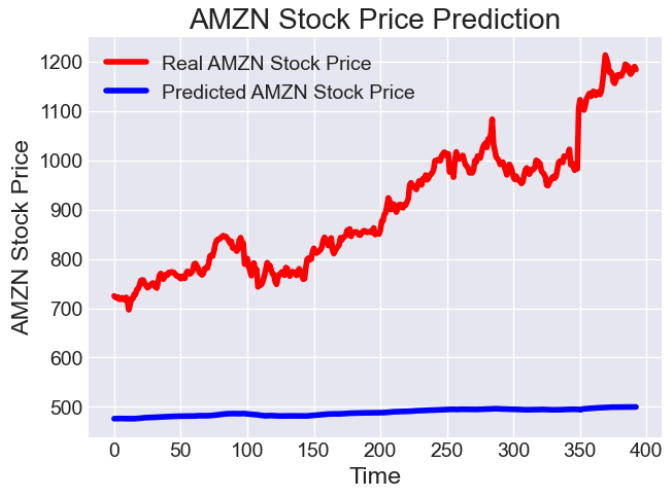

Fig. 38. Plot Prediction of the Sub-Experiment 10.3: Adding the Price Change

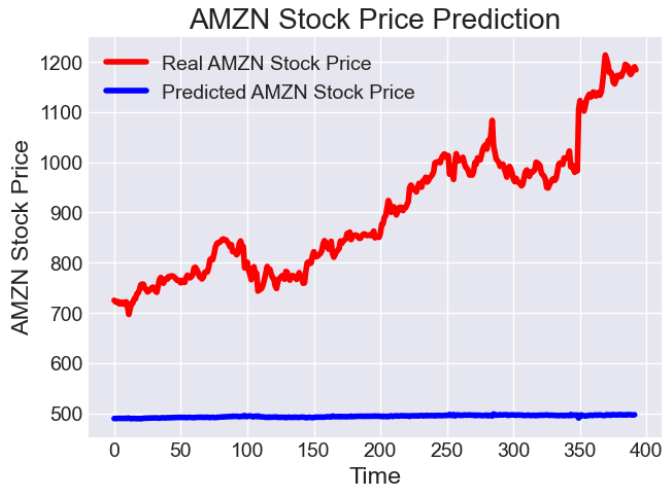

Fig. 39. Plot Prediction of the Sub-Experiment 10.4: Combining Multiple Lagged Features

TABLE 11
EXPERIMENT #10 EVALUATION METRICS VS BASELINE MODEL

EXPERIMENT	RMSE	LOSS	VALIDATION LOSS
NO LAGGED FEATURES	59.76	3.3517E-04	N/A
ADDING THE PREVIOUS DAY'S PRICE	427.07	6.2988E-04	0.0462
ADDING A MOVING AVERAGE	378.53	8.5854E-04	0.0425
ADDING THE PRICE CHANGE	431.63	9.0497E-04	0.0611
COMBINING MULTIPLE LAGGED FEATURES	426.99	6.3255E-04	0.0474

For this stock price dataset and model, simply adding these basic lagged features did not enhance the model's ability to forecast future prices. In fact, it significantly worsened the performance in all tested scenarios.

Adding the previous day's price as a feature resulted in a much higher RMSE (427.07). Figure 36 shows a predicted line that is almost flat, indicating that the model struggled to learn from this single lagged feature to predict future prices. The higher validation loss also suggests poor generalization.

Adding a 5-day moving average as a feature also led to a substantial increase in RMSE (378.53). Similar to the previous case, Figure 37 shows a very smooth predicted line that doesn't capture the actual price fluctuations well. The higher validation loss indicates a less effective model.

Adding the price change as a feature resulted in the worst performance among the single feature additions, with the highest RMSE of 431.63. Figure 38 shows a predicted line that is nearly constant. The high validation loss suggests that this feature alone was not informative for predicting future price levels.

Combining the previous day's price and the 5-day moving average did not improve the performance, resulting in an RMSE (426.99) similar to using just the previous day's price. Figure 39 still shows a flat predicted line. This suggests that simply combining these basic lagged features without more sophisticated engineering or model adjustments did not provide the model with better predictive power.

Overall, this suggests that either these specific lagged features were not highly informative on their own, or that the model architecture and training process needed to be adapted to effectively utilize these additional features. More advanced feature engineering techniques or different model architectures might be necessary to leverage lagged information successfully.

V. Conclusions

This laboratory exercise explored the impact of various modifications to a baseline LSTM neural network model for predicting Amazon stock prices. Through systematic experimentation, we investigated the effects of data scaling, network depth, recurrent unit type, number of units, learning rate, sequence length, activation functions, dropout regularization, different optimizers, and the inclusion of lagged features.

Our findings revealed that data scaling played a crucial role, with Log Transformation significantly outperforming Min-Max Scaling and z-Score Standardization, suggesting that handling the distribution of data appropriately is vital for model performance.

Regarding model architecture, increasing the number of LSTM layers beyond the baseline generally led to a degradation in performance, indicating that deeper recurrent networks, without further optimization, might overfit or struggle to learn effectively from this dataset. Comparing different recurrent units showed that the baseline LSTM architecture was superior to SimpleRNN and GRU, highlighting the importance of sophisticated gating mechanisms for capturing long-term dependencies in financial time series data.

The number of units in the LSTM layers significantly impacted performance, with deviations from the baseline of 5 units resulting in higher prediction errors. This underscores the need to find an appropriate model capacity. Similarly, the learning rate was a critical hyperparameter, with the baseline value of 0.001 proving more effective than both higher and lower rates, as well as a dynamic learning rate scheduler in this implementation.

The sequence length experiment demonstrated the importance of providing an adequate historical context to the model, with the baseline length of 60 appearing to be a reasonable balance. Both shorter and longer sequences led to decreased predictive accuracy.

The choice of activation function had a profound effect, with ReLu in the LSTM layers yielding the best results, suggesting its suitability for capturing the non-linear dynamics of stock prices. Sigmoid proved detrimental in the recurrent layers, and the output layer's activation should be chosen based on the target variable's range.

Dropout regularization, in the configurations tested, generally did not improve performance, suggesting that overfitting might not have been a primary issue or that the chosen dropout rate and placement were not optimal. Finally, the choice of optimizer was significant, with the baseline RMSprop outperforming Adam, SGD, and Adagrad, indicating its effectiveness in navigating the loss landscape for this problem.

Interestingly, the addition of simple lagged features, such as the previous day's price, moving average, and price change, did not improve the model's predictive power and often led to worse results. This suggests that more sophisticated feature engineering or model architectures might be required to effectively leverage these types of historical information.

Future work could explore several avenues for enhancing the stock price prediction model. Incorporating an additional dataset, such as the provided IBM's stock prices, alongside the existing Amazon stock price data offers a significant opportunity for advanced feature engineering. This could involve creating features that capture the correlation and co-movement between the two stocks, lagged values of IBM's price to identify potential leading or lagging indicators, or developing composite technical indicators based on both datasets. Such improvised features could provide a more comprehensive view of market dynamics influencing Amazon's stock price.

Lastly, further optimization of the Log Transformation scaling is recommended, potentially in conjunction with other transformations or hybrid approaches. A more systematic hyperparameter tuning process, employing techniques like grid search or Bayesian optimization, could be applied

to refine the number of LSTM layers and units, learning rate, and dropout rate. Experimentation with different LSTM architectures, or the inclusion of attention mechanisms, could improve the model's capacity to learn long-range dependencies. More advanced feature engineering techniques, extending beyond simple lagged features to possibly include a wider array of technical indicators or external macroeconomic factors, should be investigated. Additionally, exploring different model architectures beyond standard LSTMs, such as Transformers, which have demonstrated promise in sequence modeling, could be experimented on in the future. Finally, if learning rate schedulers are to be utilized, they require careful tuning, as default settings did not surpass a fixed learning rate in this laboratory exercise.

In conclusion, while the baseline LSTM model with appropriate scaling and hyperparameters showed a reasonable ability to predict stock prices, there are numerous avenues for further exploration and potential improvement. This laboratory exercise provides a valuable foundation for understanding the impact of various design choices in time series forecasting with recurrent neural networks.

VI. References

[1] A. Casolaro, V. Capone, G. Iannuzzo, and F. Camastra, "Deep Learning for Time Series Forecasting: Advances and Open Problems," Information, vol. 14, no. 11, p. 598, Nov. 2023, doi: https://doi.org/10.3390/info14110598.

[2] *Ajit Mohan Pattanayak, Aleena Swetapadma, and B. Sahoo, "Exploring Different Dynamics of Recurrent Neural Network Methods for Stock Market Prediction - A Comparative Study," Applied artificial intelligence, vol. 38, no. 1, Jun. 2024, doi: https://doi.org/10.1080/08839514.2024.2371706.*

[3] Batsuuri, J. (2023, September 30). Recurrent Models Overview - Computronium Blog - Medium. Medium. https://medium.com/computronium/recurrent-models-overview-d93c5bfd3376

[4] Cho, K., Bart, V. M., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014, June 3). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. arXiv.org. https://arxiv.org/abs/1406.1078

[5] DATAtab Team. (2025). DATAtab: Online Statistics Calculator. DATAtab e.U. Graz, Austria. https://datatab.net/tutorial/z-score

[6] Htoon, K. S. (2021, December 13). Log Transformation: Purpose and Interpretation - Kyaw Saw Htoon - Medium. Medium. https://medium.com/@kyawsawhtoon/log-transformation-purpose-and-interpretation-9444b4b049c9

[7] Q. Li, N. Kamaruddin, S. S. Yuhaniz, and H. A. A. Al-Jaifi, "Forecasting stock prices changes using long-short term memory neural network with symbolic genetic programming," Scientific Reports, vol. 14, no. 1, p. 422, Jan. 2024, doi: https://doi.org/10.1038/s41598-023-50783-0.

[8] *V. Varadharajan, N. Smith, D. Kalla, F. Samaah, K. Polimetla, and G. R. Kumar, "Stock Closing Price and Trend Prediction with LSTM-RNN," Ssrn.com, Feb. 15, 2024. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4728183 (accessed May 11, 2025).*